```
function merge(elem1,
elem2)                                          #1
  p1 = this.findPartition(elem1)
                                    #2
  p2 =
this.findPartition(elem2)
#3
  if p1 == p2
then                                                  #4
    return false
  this.parentsMap[p2] =
this.parentsMap[p1]                          #5
  return true
```

#1 Takes two elements, merges their partitions and returns `true` iff the two elements were in two different partitions that now are merged, `false` if they were already in the same partition;

#2-#3 Retrieve the partitions to which `elem1` and `elem2` belong to. If the argument is invalid or not found, this call will throw.

#4 Compare `p1` and `p2`, and if they are the same there is nothing left to do, the elements are already in the same partition, and so no merge happens: `false` is returned.

#5 Set the parent of `p2 to be p1`, so that now both `p1` and `p2` have the same parent, but also all elements in p2 will ultimately share `p1` as the root of their tree.

So, in summary, with this implementation we have a disjoint-set for which all the operations still require linear time in the worst case, but on average will only need logarithmic time: even for those operations that were constant-time in the naïve implementation… Admittedly, that doesn't sound a